

Notas de Aula – Guilherme Sipahi – Arquitetura de Computadores

Conjunto de Instruções:

Elementos de uma instrução:

- *Operation Code (Opcode)*: especifica a operação a ser executada.
- *Source operand reference*: a operação pode envolver um ou mais operandos de entrada.
- *Result -operand reference*: o resultado que pode ser reproduzido pela operação.
- *Referência a nova instrução*: conta à CPU onde buscar a nova operação depois de concluída a execução do programa.

Na maior parte dos casos a próxima instrução segue a instrução que está sendo executada na memória. Nestes casos não há referência explícita à próxima instrução. Quando uma referência explícita é necessária então o endereço na memória (principal ou virtual) deve ser passado.

Os operandos podem se encontrar em três áreas distintas:

- *Memória principal ou virtual*.
- *Registro da CPU*: com raras exceções uma CPU contém um ou mais registros que podem ser referenciados pelas instruções de máquina. Se for apenas um registro a referência é implícita.
- *Dispositivo de Entrada e Saída*.

Representação de uma Instrução:

Opcode (4 bits) | Referencia Operando (6 bits) | Referencia Operando (6 bits)

Mnemônicos (facilitam o entendimento das instruções de máquina)

ADD	Add
SUB	subtract
MPY	multiply
DIV	divide
LOAD	load data from memory
STOR	store data to memory.

Exemplo de instrução:

ADD R, Y → endereço de memória.
↓
registrador

Tipos de Instrução:

Linguagem de alto nível:

$X = X + Y$ $X \rightarrow 513$ (endereço de memória)
 $Y \rightarrow 514$

1. Preenchimento do registro com o conteúdo de endereço 513.
2. Adiciona o conteúdo do endereço 514 ao registro.
3. Guarde o conteúdo do registro no endereço 513.

Por este exemplo pode-se ver que uma instrução simples em linguagem de alto nível pode necessitar de três instruções de máquina.

Categorias de tipos de instrução:

- Processamento de dados: instruções lógicas e aritméticas.
- Salvamento de dados: instruções de memória.
- Movimento de dados: instruções de entrada e saída.
- Controle: teste e desvio.

Número de endereços:

Examinando os endereços necessários a uma instrução:

- 1 ou dois operandos;
- endereço do resultado;
- 1 endereço para a próxima instrução.

Quatro endereços é extremamente raro. O endereço de instrução subsequente é obtido do contador do programa.

Instruções com zero endereços => pilha de memória.

Código:

$$Y = (A - B) / (C + D \times E)$$

Instruções de 1 endereço necessitam de um registro para acumular o resultado, este é um endereço implícito (AC).

Representação Simbólica:

Número de Endereços	Representação	Interpretação
3	OP A,B,C	$A \leftarrow B \text{ OP } C$
2	OP A,B	$A \leftarrow A \text{ OP } B$
1	OP A	$AC \leftarrow AC \text{ OP } A$
0	OP	$T \leftarrow T \text{ OP } (T-1)$

T : topo da pilha.

O número de instruções é uma decisão fundamental para o design. Menos endereços requerem uma CPU mais simples e têm instruções mais curtas. Porém o

programa conterá mais instruções.

Um endereço → AC → um único registro.

Múltiplos endereços → múltiplos registros → operações podem ser executadas apenas em registros → mais rígidas.

Design do conjunto de instruções.

Define muitas das funções da CPU e portanto é muito importante na implementação da mesma.

- Repertório de operações.
- Tipos de Dados.
- Formato das Instruções.
- Registros.
- Endereçamento.

Tipos de Operandos.

- Endereços → unsigned integer;
- Números → inteiros, pontos flutuantes e decimais;
- Caracteres → IRA (International Reference Alphabet) ou ASCII (American Standard Code for Information Interchange.)
- Lógicos → booleans.

a) Endereços:

São uma forma de dado. Pode ser necessário fazer cálculos para determinar o endereço real ou virtual - nesse contexto endereços podem ser considerados "unsigned integers"

b) Números:

Qualquer linguagem de máquina inclui números, mesmo para processamento não-numérico.

Três tipos de números são comuns:

- Inteiros;
- "Floating point"
- Decimais: são usados por não haver necessidade de conversão de binário para decimal quando:

Packed decimal:

- cada dígito decimal é representado por um código de 4 bits.
- É menos compacto e menos eficiente que o código binário: apenas 10 dos 16 valores são utilizados.
- Diversas máquinas provêm instruções aritméticas para operarem diretamente com números packed decimal.

c) Caracteres:

Caracteres não podem ser armazenados ou transmitidos por meios digitais. Somente números binários são próprios para ser utilizados em computadores => É necessário criar códigos para representar caracteres por uma sequência de bits.

- International Reference Alphabet (IRA) ou mais conhecido como: American Standard Code for Information Interchange (ASCII):

Código de 7 bits => 128 caracteres.

- Caracteres imprimíveis;
- Caracteres de controle, não imprimíveis que controlam a impressão ou tem a ver com a comunicação de dados;
- oitavo bit => zeros ou parity bit pra detecção de erro.

(números de "11" no octeto é par (paridade par) ou ímpar (paridade ímpar)

números quaisquer são representados pelos binários => conversão fácil para packed decimal)

- Extended Binary Coded Decimal Interchange Code:

EBCDIC - 8 bits

Packed decimal 1111 0000 - 11111001 (0 - 9)

Data Types:

Pentium II

- byte (8 bits)
- word (16 bits)
- double word (32 bits)
- quadword (64 bits)

Não há necessidade de alinhamento em endereços pares para palavras (word) nem com endereço divisíveis por quatro para double word, etc ...

utilização de memória mais eficiente.

O bus (32 bits) porém transmite dados em unidades possíveis por 4. O processador converte as requisições de valores desalinhados em uma sequência de requisições para uma transferência do bus.

Pentium II usa little endian

- Floating => IEEE754 standard conforme (full)
- Inteiros => Complemento de 2 (16, 32 ou 64 bits)
- Operações de floating => operam em inteiros, floating, unpacked decimal.
123456789

little endian => 78 56 34 12

big endian => 12 34 56 78

Dados Lógicos:

Normalmente uma unidade endereçável (byte, palavra, etc...) é tratada como uma unidade de dado.

Pode ser útil tratar esta unidade como 1 - bit ítems de dados. => Dados lógicos.

Utilização:

- análise de dados "booleanos";

- Manipulação de bits de uma unidade de dados:
 -) deslocar bits em uma operação de ponto flutuante.
 -) conversão de ASCII para packed decimal => é necessário extrair os 4 bits da direita.

Observação:

- Numa linguagem de máquina o tipo de dado da unidade de dados é normalmente definido pela operação que é executada nele.
- Em linguagens de alto nível você define o tipo de dados de que se trata a unidade de dados.

Power PC:

- byte (8 bits)
- half word (16 bits)
- word (32 bits)
- double word (64 bits)

Algumas instruções requerem que os operandos da memória sejam alinhados em 32 bits.

No entanto, em geral, o alinhamento não é necessário.

Power PC pode usar little endian ou big endian.

- Byte, half-word, word e double word são tipos de dados gerais.
- Unsigned byte e half word operações lógicas e aritméticas: ficam à direita e preenchem os bits à esquerda com zeros.
- Unsigned word: operações, aritméticas e ponteiro de endereços.
- Signed half-word: operações aritméticas preenche valores à esquerda com o bit de sinal.
- Signed word: operações aritméticas.
- Unsigned double word: ponteiro de endereços.
- Byte string: 0 a 128 bits de comprimento.

IEEE standard conforme para:

- Floating point de precisão simples.
- Floating point de dupla precisão.

Tipos de Operação (classificação típica):

- Transferência de Dados
- Aritméticas
- Lógicas
- Conversões
- I/O
- Controle do Sistema
- Transferência de controle

Tabela 9.3

Examinaremos um pouco mais detalhadamente o que cada uma dessas operações precisa processar:

- **Transferência de Dados:**

É necessário determinar:

- A posição dos operandos (fonte e destino). Estes podem estar na memória, num registro ou no topo de uma pilha
- O tamanho do dado a ser transferido
- O modo de endereçamento de cada operando

tabela 9.5

O que a CPU faz?

- Se a fonte e o destino são registros: Transfere os dados de um registro ao outro.
- Se algum dos operandos está na memória :
 - Calcula o endereço, baseado no modo de endereçamento.
 - Se o endereço está na memória virtual traduz o endereço para a memória real.
 - Determina se o item está no cachê.
 - Se não, envia o comando ao modulo de memória.
- **Aritméticos**

A maioria das maquinas provê as operações básicas de somar, subtrair, multiplicar e dividir inteiros. Alguns provem essas operações para floating e decimais compactos.

Outras operações possíveis:

 - Absoluto: Tem como resultado o valor absoluto do numero
 - Negação: Inverte o valor d operando
 - Incremento: incrementa o operando em 1
 - Decremento: decrementa o operando em 1

- **Lógicos**

- NOT: inverte um bit
- AND: um e outro
- OR: qualquer dos dois
- XOR: um ou outro
- EQUAL:

AND em conjunto com uma mascara de 111 para as posições desejadas pode eliminar pedaços indesejados da palavra. Exemplo:

- 2 caracteres por palavra (16 bits)
- quer mandar um por vez

11111111 00000000	⇒	10011010 00000000
10011010 11011000		
10011010 11011000	⇒	00000000 11011000
00000000 11111111		

Outras operações lógicas:

Shifts e rotates: Figura 9.5

Shift lógico: move todos os bits e adiciona zero as bordas

Shift aritmético: deixa o sinal intocável

Para o shift à esquerda, o bit da direita é zero

Para o shift à direita, o 2º bit é o bit do sinal

Rotates:

À direita ⇒ divisão por dois

À esquerda ⇒ multiplicação por dois

- **Conversões**

Através de tabelas converte, por exemplo EBCDIC para ASCII

- **I/O**

- **Controle do sistema**

Em geral reservadas para serem usadas pelo S.O.

- **Transferência de Controle**

Até agora, as operações discutidas tem uma sequência bem definida, uma instrução é seguida na execução pela instrução que se encontra em um endereço subsequente.

Há pelo menos 3 motivos para executar instruções fora de ordem.

- Com 'loops' podem ser simplificados programas de tarefas repetitivas
- Tomadas de decisões necessitam de desvios
- Chamadas a sub-rotinas facilitam a programação

Três tipos de desvios:

4. Branch

BRP X

BRN X ⇒ refere-se ao resultado anterior, ou

BRZ X

BRO X

BRE R1,R2,X ⇒ vai para X se R1=R2 (Figura 9.6)

5. Skip

Loops podem também serem feitos com skips

301



309 desvios não condicionais e 'skips' também são muito comuns

310

311

6. Sub-rotinas

Motivos principais: economia e modularidade

O código não precisa ser replicado e as rotinas podem ser desenvolvidas por pessoa diferentes ou mesmo que seja pela mesma pessoa, tornando mais inteligível o programa.

CALL e RETURN ⇒ mostrar figura 9.7

É importante notar que:

- a sub-rotina pode ser chamada de mais de uma posição
- uma sub-rotina pode ser chamada por outra ⇒ 'nesting' (aninhamento) de profundidade arbitrária
- CALL pode ser casado com um RETURN ao final da sub-rotina

Três possibilidades para guardar o ponto de retorno:

- registro: uso de um registro para guardar o endereço da próxima instrução
- início da sub-rotina: guarda o endereço da próxima instrução em uma variável dentro da sub-rotina, executando esta instrução logo que se inicia a mesma
- pilhas: a vantagem da pilha é que ela permite reentrâncias e seu caso específico recorrência

figura 9.8

[1] William Stallings - Computer Architecture and Organization, Pearson, 5th Edition